L0_Lidar.lua

```
1
2  Lidar = {
3      A = nil,      -- Input target angle, must be kept constant
4      r = nil,      -- Input request obstacle distance
5      E = nil,      -- Input obstacle distance
6      T = nil,      -- Time [s]
7      dT = nil,     -- Time step [s]
8      _wrot = nil, -- Sensor servo speed in degrees per cyle
9      state = {},
10     B = {},       -- Begin time
11     D = {},       -- Delay
12     Y = {},       -- Yaw
13     Z = {},       -- Control variable for yaw Y
14     Q = {},       -- Output obstacle distance
15     p = {},       -- Pending request
16     u = {},       -- Distance update
17     s = false,    -- Send ping
18     Y2Z = function(A)
19       if A==360 then A=0 else A=math.floor(1500-A*160/90) end
20       return A
21     end, -- function Y2Z
22     forward = function(self)
23       self.state.curr = self.state.next
24       self.B.curr = self.B.next
25       self.D.curr = self.D.next
26       self.Y.curr = self.Y.next
27       self.Z.curr = self.Z.next
28       self.Q.curr = self.Q.next
29       self.p.curr = self.p.next
30       self.u.curr = self.u.next
31       self.s = (self.state.curr=="PING")
32     end, -- function forward()
33     init = function(self)
34         self._wrot = 120; -- deg/s (aprox. half of reality)
35         self.T = sim.getSimulationTime()
36         self.state.prev = "NONE" -- for use in cmonitor() only
37         self.state.next = "ROTATE"
38         self.B.next = self.T
39         self.D.next = 90/self._wrot
40         self.Y.next = 0
41         self.Z.next = self.Y2Z(0)
42         self.Q.next = 0
43         self.p.next = false
44         self.u.next = false
45         self.s = false
46         self:forward()
47     end, -- function init()
48     monitor = function(self)
49         print(string.format("Lidar: state=%s p=", self.state.curr))
50         if self.p.curr then print("true") else print("false") end
51         print(string.format(" B= %.4fs D= %.4fs Y= %ideg Z=%ius Q=%icm u=",
52             self.B.curr, self.D.curr, self.Y.curr, self.Z.curr, self.Q.curr))
53         if self.u.curr then println("true") else println("false") end
54     end, -- function monitor()
55     cmonitor = function(self)
56         if self.state.prev~=self.state.curr then
57             self.state.prev = self.state.curr; self.monitor(self)
58         end -- if
59     end, -- function cmonitor()
60     read_inputs = function(self, A, r)
61         if A~=nil then
62             if A<-90 then self.A = -90
63             elseif A>90 then self.A = 90
64             else self.A = A
65             end -- if chain
66         else
67             self.A = 0
68         end -- if
69         if r~=nil then self.r = r else self.r = false end
70         local echo = sim.getInt32Signal(sensor_echo)
71         if echo==nil or echo<0 then
72             self.E = nil
73         else
74             self.E = echo -- [cm]
75         end -- if
76         self.T = sim.getSimulationTime()
77         self.dT = sim.getSimulationTimeStep()
78         --print(string.format("@%.4fs: L0Lidar.E = ", self.T))
79         --if self.E then println(string.format("%d", echo)) else println("nil") end
80     end, -- function read_inputs()
81     step = function(self)
82       if self.state.curr=="ROTATE" then
83         self.p.next = self.p.curr or self.r -- in case some request interrupts a previous
one
84         if self.T-self.B.curr>=self.D.curr then
85           self.Z.next = self.Y2Z(360)
86           self.state.next = "PING"
87         end -- if
88       elseif self.state.curr=="PING" then
89         self.p.next = self.p.curr or self.r
90         if self.A~=self.Y.curr then
91           self.B.next = self.T
92           self.D.next = math.abs(self.A-self.Y.curr)/self._wrot
93           self.Y.next = self.A
94           self.Z.next = self.Y2Z(self.A)
95           self.u.next = false
96           self.state.next = "ROTATE"
97         else
98           self.state.next = "ECHO"
99         end -- if chain
100      elseif self.state.curr=="ECHO" then
101        if self.r then
102          self.p.next = true
103          self.state.next = "PING"
104        elseif self.E~=nil then
105          self.p.next = false
106          self.u.next = self.p.curr
107          self.Q.next = self.E
108          self.state.next = "PING"
109        end -- if
110      else -- Stop state or error
111      end -- if chain
112    end, -- function step()
113    write_outputs = function(self)
114      if self.s then
115        sim.setInt32Signal(sensor_ping, 1)
116      else
```

```
        sim.setInt32Signal(sensor_ping, 0)
      end
      if self.Z.curr~=360 then sim.setInt32Signal(servo_sensor, self.Z.curr) end
    end, -- function write_outputs()
    get_distance = function(self)
        --println("L0_Lidar.get_distance="..self.Q.curr)
        return self.Q.curr
    end, -- function get_distance()
    get_update = function(self)
        --[[if self.u.curr then
          println("L0_Lidar.get_update=true")
        else
          println("L0_Lidar.get_update=false")
        end--]]
        return self.u.curr
    end, -- function get_update()
    active = function(self)
        return (self.state.curr=="ROTATE") or
               (self.state.curr=="PING") or
               (self.state.curr=="ECHO")
    end -- function active()
} -- Lidar
```